SCHOOL MANAGEMENT SYSTEM FOR KYAMATU PRIMARY SCHOOL

SYSTEM DESIGN SPECIFICATION

Mwange Musa Muthami

Registration Number: 23/05037

Submission Date: April 2026

Supervisor: Dr. Simon N. Mwendia / Dr. Kevin Mugoye Sindu

A project document submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Information Technology (BIT 3105)

1. SYSTEM ARCHITECTURE

1.1 Three-Tier Architecture

The system uses a three-tier client-server architecture:

Tier	Technology	Purpose
Client	React 18 + Vite + Tailwind CSS	Single Page Application UI
Server	Node.js 20 + Express.js	RESTful API backend
Database	PostgreSQL 15 + Prisma ORM	Relational data storage

1.2 Deployment Architecture

Frontend: Cloudflare Pages (auto-deploy from GitHub)

Backend: Render (auto-deploy from GitHub, Port 3000)

Database: Supabase (Managed PostgreSQL)

2. DATABASE DESIGN

2.1 Entity-Relationship Overview

The database contains 23+ tables with UUID primary keys. Key relationships:

- User → Student, Staff, Guardian (one-to-one)
- Student → Class (many-to-one), Attendances, Scores, Payments (one-to-many)
- Class → Grade, Stream, AcademicYear (many-to-one)
- Assessment → Subject, Term (many-to-one), Scores (one-to-many)
- StudentInvoice → Payments, InvoiceItems (one-to-many)

2.2 Key Tables

Table	Purpose	Key Fields
User	Authentication accounts	id, email, password, role
Student	Learner profiles	admissionNumber, firstName, classId, admissionStatus
Staff	Teacher/staff profiles	employeeNumber, firstName, qualification
Class	Grade+Stream+Year combo	name, gradeId, streamId, academicYearId
Attendance	Daily records	date, status, studentId, classId
Assessment	Test/exam definitions	name, type, maxScore, weight, subjectId
AssessmentScore	Student scores	score, grade, studentId, assessmentId
ReportCard	Termly reports	totalScore, averageScore, rank
FeeStructure	Fee configuration	name, amount, gradeId, termId

StudentInvoice	Billing	invoiceNo, totalAmount, paidAmount, balance
Payment	Transactions	amount, method, status, transactionRef

3. MODULE DESIGN

3.1 Backend Feature Module Pattern

Each feature follows a three-file pattern: routes.js (route definitions + validation), controller.js (request handling), service.js (business logic + DB queries).

Feature modules: auth, students, staff, academic, attendance, assessments, reports, fees, communication, dashboard.

3.2 Middleware Pipeline

Order	Middleware	Purpose
1	CORS	Cross-origin handling
2	Helmet	Security headers
3	Rate Limiter	Request throttling
4	Request ID	Unique request tracking
5	Body Parser	JSON parsing
6	Sanitize	Input sanitization
7	Auth (JWT)	Token verification
8	RBAC	Role permission check
9	Validate	Request validation

3.3 Authentication Flow

1. Client sends POST /auth/login with email and password.
2. Server validates credentials with bcrypt.compare().
3. Server generates JWT access token (15 min) and refresh token (7 days).
4. Client stores tokens and includes access token in Authorization header.
5. Server verifies JWT and checks RBAC permissions on each request.

4. FRONTEND DESIGN

4.1 Technology Stack

Technology	Purpose
React 18	Component-based UI
Vite	Build tool
Tailwind CSS	Styling
Zustand	State management
Axios	HTTP client
React Router	Client-side routing
Recharts	Data visualization
Lucide React	Icons

4.2 Page Structure (17+ Pages)

- Login – Authentication form
- Dashboard – Role-specific analytics
- Students – Student CRUD with search/filter
- Admissions – Approval/rejection workflow
- Staff – Staff management
- Classes – Grade/stream management
- Attendance – Daily marking interface
- Assessments – Score entry
- Reports – Report card generation
- Fees – Financial management
- Timetable – Weekly grid view
- Announcements – Communication
- Settings – System configuration
- Profile – User profile
- System Status – Health monitoring

5. SECURITY DESIGN

Aspect	Implementation
Authentication	JWT (HS256), 15min access + 7day refresh tokens
Password Hashing	bcrypt with 12 salt rounds
Authorization	RBAC middleware per route
Input Validation	express-validator on all fields
Sanitization	Custom middleware strips HTML/scripts
Rate Limiting	Redis-backed request limits per IP
Security Headers	Helmet.js (XSS, CSRF, clickjacking)
Audit Trail	All critical operations logged